

Arquitectura de Datos

Tema 3: Redis

Bases de datos clave-valor en memoria

Curso académico 2026/2027



Parte 1 Patrón Cache-Aside



Objetivo

Implementar el patrón **Cache-Aside** (también conocido como *Lazy Loading*) utilizando **Redis** como capa de caché, **JetBrains DataGrip** para simulación de base de datos, **C#** con StackExchange.Redis y **Python** con redis-py.

Al finalizar este laboratorio serás capaz de:

- Comprender y aplicar el patrón Cache-Aside en escenarios reales.
- Simular una base de datos relacional en Redis usando estructuras Hash.
- Implementar Cache-Aside desde DataGrip con comandos Redis directamente.
- Desarrollar una implementación completa en C# con StackExchange.Redis.
- Desarrollar una implementación alternativa en Python con redis-py.
- Implementar invalidación de caché mediante el comando DEL.

Requisitos previos

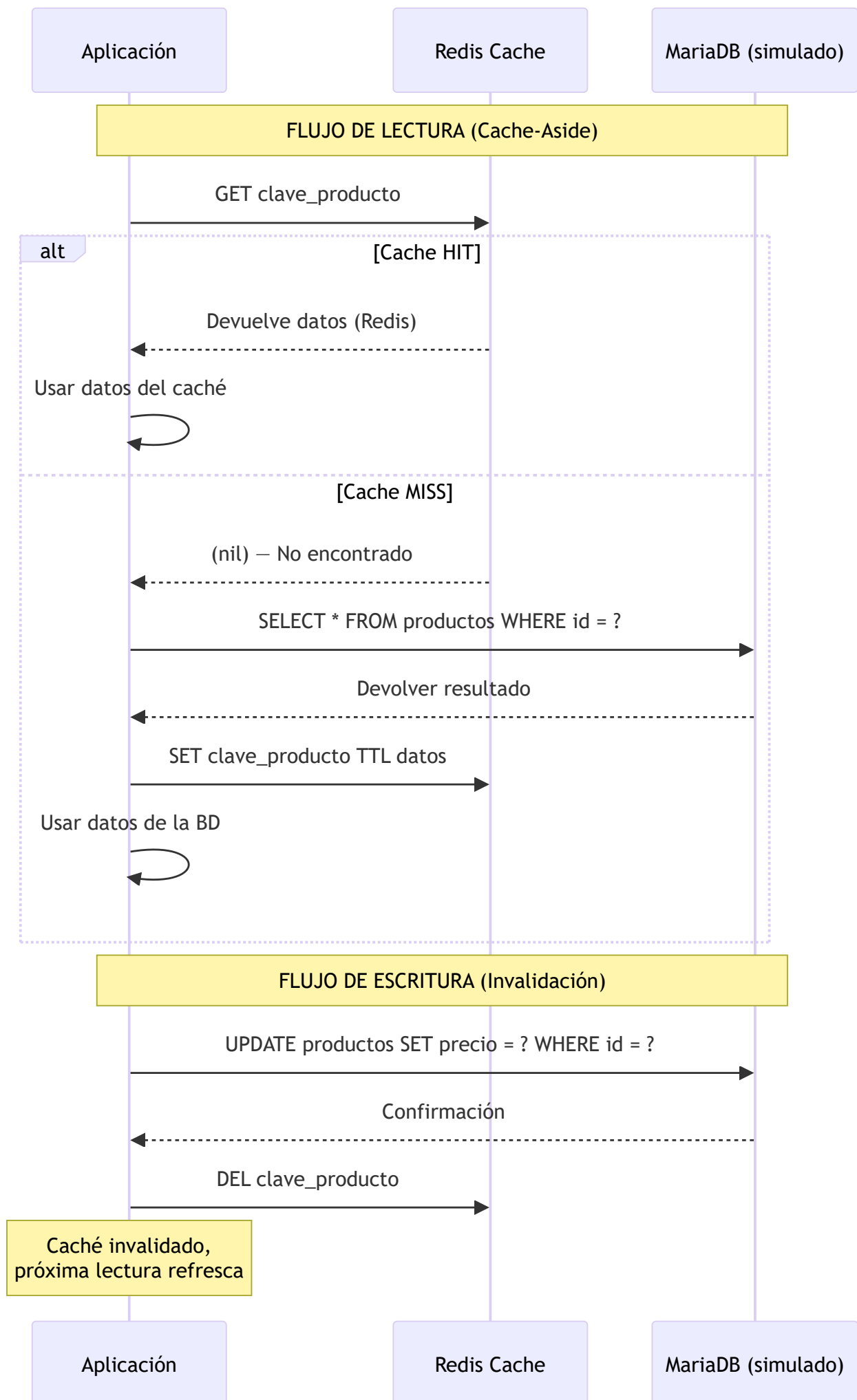
Componente	Versión mínima	Comprobación
Docker Desktop	4.x	<code>docker --version</code>
JetBrains DataGrip	2024.x	Abrir la aplicación
.NET SDK	10.0	<code>dotnet --version</code>
Python	3.12+	<code>python --version</code>
Práctica PR1 completada	—	Redis corriendo en Docker
Práctica PR2 completada	—	Estructuras de datos dominadas

Nota: Asegúrate de que el contenedor Redis de la Práctica PR1 está en ejecución antes de comenzar. Si usas Windows, verifica que Docker Desktop está activo.

Instalación de .NET 10: Si no tienes .NET SDK instalado, descárgalo desde <https://dotnet.microsoft.com/download/dotnet/10.0>. Selecciona tu sistema operativo (Windows, Linux o macOS) y sigue las instrucciones del instalador.

Arquitectura del laboratorio

El siguiente diagrama de secuencia muestra el flujo completo del patrón Cache-Aside, incluyendo tanto la lectura como la escritura:



Explicación: En Cache-Aside, la aplicación es responsable de gestionar la caché. Primero busca en Redis; si no encuentra el dato (MISS), lo consulta en la base de datos y lo almacena en caché para futuras peticiones. Al modificar datos en la BD, invalida la caché para mantener la coherencia.

Paso 1: Simular MariaDB en Redis

Antes de implementar Cache-Aside, necesitamos simular una base de datos de productos en Redis. Utilizaremos la estructura **Hash** para representar registros de una tabla, ya que permite almacenar múltiples campos por clave.

1.1 Crear tres productos con Hash

Cada producto se almacena como un Hash con campos que simulan columnas de una tabla relacional: id, nombre, precio y stock.

Redis CLI – Crear producto 1

```
# Producto 1: Portátil gaming
# Cada campo simula una columna de la tabla "productos"
HSET producto:1 id 1 nombre "Portátil Gaming XYZ" precio 1299.99 stock 15
HSET producto:1 descripcion "Portátil de alta gama con RTX 4070"
```

Salida esperada:

Resultado

```
(integer) 4
(integer) 1
```

Redis CLI – Crear producto 2

```
# Producto 2: Monitor 4K
HSET producto:2 id 2 nombre "Monitor 4K 27\"" precio 449.95 stock 32
HSET producto:2 descripcion "Monitor IPS 27 pulgadas resolución 4K"
```

Salida esperada:

Resultado

```
(integer) 4
(integer) 1
```

```
# Producto 3: Teclado mecánico
HSET producto:3 id 3 nombre "Teclado Mecánico RGB" precio 89.90 stock 67
HSET producto:3 descripcion "Teclado switches Cherry MX Brown"
```

Salida esperada:

Resultado

```
(integer) 4
(integer) 1
```

1.2 Verificar los productos creados

Redis CLI – Consultar todos los campos

```
# Obtener todos los campos y valores del producto 1
HGETALL producto:1
```

Salida esperada:

Resultado

```
"id" → "1" → "nombre" → "Portátil Gaming XYZ" → "precio" → "1299.99" → "stock" → "15" → "desc
```

1.3 Verificar existencia de productos

Redis CLI – Verificar claves

```
# Verificar que los tres productos existen
EXISTS producto:1 producto:2 producto:3
```

Salida esperada:

Resultado

```
(integer) 3
```

¿Por qué Hash? La estructura Hash permite representar registros de tablas de forma natural. Cada campo del Hash equivale a una columna, y cada clave producto:N equivale a una fila. Es más eficiente que almacenar JSON como string porque puedes actualizar campos individuales sin reescribir todo el objeto.

Paso 2: Cache-Aside con DataGrip

En este paso implementaremos el patrón Cache-Aside manualmente utilizando la consola de DataGrip o Redis CLI. Simularemos el flujo completo de lectura con cache miss y cache hit.

2.1 Primer acceso — Cache MISS

Imagina que la aplicación busca el producto 2 por primera vez. No existe en caché, por lo que consultamos la "base de datos" (simulada en Redis) y almacenamos el resultado con expiración.

Redis CLI — Simular cache miss

```
# PASO 1: Buscar en caché (simula GET de la aplicación)
# Usaremos una clave con prefijo "cache:" para diferenciar caché de BD
GET cache:producto:2
```

Salida esperada: (cache miss)

Resultado

```
(nil)
```

Redis CLI — Consultar la "base de datos"

```
# PASO 2: Cache miss – Consultar la base de datos (simulada)
HGETALL producto:2
```

Salida esperada:

Resultado

```
"id" → "2" → "nombre" → "Monitor 4K 27\" → "precio" → "449.95" → "stock" → "32" → "descripci
```

Redis CLI — Almacenar en caché con TTL

```
# PASO 3: Almacenar el resultado en caché con TTL de 300 segundos
# Serializamos el Hash como JSON para simplificar la caché
SETEX cache:producto:2 300 '{"id":2,"nombre":"Monitor 4K 27\"
```

Salida esperada:

Resultado

OK

2.2 Secondo accesso — Cache HIT

La siguiente vez que se solicita el mismo producto, Redis lo devuelve directamente desde la caché sin necesidad de consultar la "base de datos".

```
Redis CLI - Simular cache hit
```

```
# PASO 1: Buscar en caché (cache hit esta vez)
GET cache:producto:2
```

Salida esperada: (cache hit)

Resultado

```
"{"id\":"2","\nombre\":"Monitor 4K 27\\\\"","\precio\":"449.95","\stock\":"32}"
```

2.3 Verificar TTL del caché

Redis CLI – Comprobar tiempo restante

```
# Consultar el TTL restante del caché
TTL cache:producto:2
```

Salida esperada: (el valor variará)

Resultado

```
(integer) 287
```

¿Por qué TTL? El tiempo de vida (TTL) garantiza que los datos en caché no queden obsoletos indefinidamente. Si la base de datos se actualiza, el caché expirará automáticamente y la próxima lectura obtendrá datos frescos. Un TTL de 300 segundos (5 minutos) es razonable para datos de catálogo de productos.

Paso 3: Implementación en C# con StackExchange.Redis

En este paso implementaremos el patrón Cache-Aside completo en C# utilizando la librería **StackExchange.Redis**, el cliente Redis más popular para .NET.

3.1 Crear el proyecto

Terminal – Crear proyecto

```
# Crear directorio del proyecto
mkdir CacheAsideDemo
cd CacheAsideDemo

# Crear proyecto .NET 10 de consola
dotnet new console -n CacheAsideDemo

# Añadir la librería StackExchange.Redis
dotnet add package StackExchange.Redis
```

3.2 Código completo de la implementación

Reemplaza el contenido de Program.cs con el siguiente código:


```
// .NET 10 + C# 14: Top-Level statements
// Sin class Program, sin Main() – el código se ejecuta directamente
using StackExchange.Redis;
using System.Text.Json;

// Conexión Redis – se reutiliza para todas las operaciones
using var redis = await ConnectionMultiplexer.ConnectAsync("localhost:6379");
var database = redis.GetDatabase();

// TTL del caché: 5 minutos (en segundos)
const int CacheTTL = 300;

Console.WriteLine("=== Cache-Aside Pattern Demo ===\n");

// Simular la base de datos de productos
var productosDb = new Dictionary<int, Producto>
{
    { 1, new Producto { Id = 1, Nombre = "Portátil Gaming XYZ", Precio = 1299.99, Stock = 15 } },
    { 2, new Producto { Id = 2, Nombre = "Monitor 4K 27\"", Precio = 449.95, Stock = 32 } },
    { 3, new Producto { Id = 3, Nombre = "Teclado Mecánico RGB", Precio = 89.90, Stock = 67 } }
};

// Ejemplo 1: Cache MISS – primera consulta
Console.WriteLine("--- Ejemplo 1: Primera consulta (Cache MISS) ---");
var producto1 = await ObtenerProductoConCacheAsync(2);
Console.WriteLine($"Producto obtenido: {producto1?.Nombre} – {producto1?.Precio}€\n");

// Ejemplo 2: Cache HIT – segunda consulta
Console.WriteLine("--- Ejemplo 2: Segunda consulta (Cache HIT) ---");
var producto2 = await ObtenerProductoConCacheAsync(2);
Console.WriteLine($"Producto obtenido: {producto2?.Nombre} – {producto2?.Precio}€\n");

// Ejemplo 3: Invalidación de caché
Console.WriteLine("--- Ejemplo 3: Invalidación de caché ---");
await InvalidarCacheProductoAsync(2);
Console.WriteLine("Caché del producto 2 eliminado.\n");

// Ejemplo 4: Después de invalidar, se produce un nuevo cache miss
Console.WriteLine("--- Ejemplo 4: Tras invalidar (Cache MISS) ---");
var producto4 = await ObtenerProductoConCacheAsync(2);
Console.WriteLine($"Producto obtenido: {producto4?.Nombre} – {producto4?.Precio}€\n");

Console.WriteLine("=== Demo completada ===");

// -----
// Funciones locales (reemplazan los métodos estáticos de la clase)
// -----

async Task<Producto?> ObtenerProductoConCacheAsync(int productoId)
{
```

```

string claveCache = $"cache:producto:{productoId}";

// PASO 1: Buscar en caché
string? cached = await database.StringGetAsync(claveCache);

if (!string.IsNullOrEmpty(cached))
{
    // CACHE HIT: deserializar y devolver
    Console.WriteLine($" [CACHE HIT] Producto {productoId} encontrado en Redis");
    return JsonSerializer.Deserialize<Producto>(cached);
}

// CACHE MISS: consultar la base de datos
Console.WriteLine($" [CACHE MISS] Producto {productoId} no está en caché. Consultando BD");
await Task.Delay(50); // Simular latencia de consulta a BD

if (!productosDb.TryGetValue(productoId, out var producto))
{
    Console.WriteLine($" [ERROR] Producto {productoId} no existe en la BD");
    return null;
}

// Almacenar en caché con TTL
string serialized = JsonSerializer.Serialize(producto);
await database.StringSetAsync(claveCache, serialized, TimeSpan.FromSeconds(CacheTTL));
Console.WriteLine($" [CACHE SET] Producto {productoId} almacenado en Redis (TTL: {CacheTTL} segundos)");

return producto;
}

async Task InvalidarCacheProductoAsync(int productoId)
{
    string claveCache = $"cache:producto:{productoId}";
    await database.KeyDeleteAsync(claveCache);
}

// -----
// Modelo de datos (declaraciones de tipos al final en top-level)
// -----
public class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double Precio { get; set; }
    public int Stock { get; set; }
}

```

3.3 Ejecutar el ejemplo

Terminal – Ejecutar

```
# Ejecutar el programa
dotnet run
```

Salida esperada:

Salida

```
=== Cache-Aside Pattern Demo ===

--- Ejemplo 1: Primera consulta (Cache MISS) ---
[CACHE MISS] Producto 2 no está en caché. Consultando BD...
[CACHE SET] Producto 2 almacenado en Redis (TTL: 300s)
Producto obtenido: Monitor 4K 27" – 449.95€

--- Ejemplo 2: Segunda consulta (Cache HIT) ---
[CACHE HIT] Producto 2 encontrado en Redis
Producto obtenido: Monitor 4K 27" – 449.95€

--- Ejemplo 3: Invalidación de caché ---
Caché del producto 2 eliminado.

--- Ejemplo 4: Tras invalidar (Cache MISS) ---
[CACHE MISS] Producto 2 no está en caché. Consultando BD...
[CACHE SET] Producto 2 almacenado en Redis (TTL: 300s)
Producto obtenido: Monitor 4K 27" – 449.95€

=== Demo completada ===
```

StackExchange.Redis: Esta librería gestiona automáticamente el pool de conexiones y reconexiones. La instancia `ConnectionMultiplexer` debe ser reutilizada (singleton) para evitar agotar los archivos de socket del sistema operativo.

Paso 4: Alternativa en Python con redis-py

A continuación se presenta una implementación equivalente en **Python** utilizando la librería `redis-py`. Esta es una alternativa para quienes prefieren Python sobre C#.

4.1 Instalar dependencias

Terminal – Instalar redis-py

```
# Crear entorno virtual
python -m venv venv

# Activar entorno (Windows)
.\venv\Scripts\activate

# Activar entorno (Linux/Mac)
# source venv/bin/activate

# Instalar redis-py
pip install redis
```

4.2 Código completo de la implementación

cache_aside.py – Implementación Cache-Aside

```
import json
import time
import redis

# Conexión Redis – se reutiliza para todas las operaciones
r = redis.Redis(host="localhost", port=6379, decode_responses=True)

# TTL del caché: 5 minutos (en segundos)
CACHE_TTL = 300

# Simulación de la base de datos de productos
PRODUCTOS_DB = {
    1: {"id": 1, "nombre": "Portátil Gaming XYZ", "precio": 1299.99, "stock": 15},
    2: {"id": 2, "nombre": "Monitor 4K 27\"", "precio": 449.95, "stock": 32},
    3: {"id": 3, "nombre": "Teclado Mecánico RGB", "precio": 89.90, "stock": 67},
}

def obtener_producto_con_cache(producto_id: int) -> dict | None:
    """
    Implementación del patrón Cache-Aside:
    1. Busca en Redis (caché).
    2. Si no existe, consulta la base de datos.
    3. Almacena el resultado en caché con TTL.
    """
    clave_cache = f"cache:producto:{producto_id}"

    # PASO 1: Buscar en caché
    cached = r.get(clave_cache)

    if cached:
        # CACHE HIT: deserializar y devolver
        print(f" [CACHE HIT] Producto {producto_id} encontrado en Redis")
        return json.loads(cached)

    # CACHE MISS: consultar la base de datos
    print(f" [CACHE MISS] Producto {producto_id} no está en caché. Consultando BD...")
    time.sleep(0.05) # Simular latencia de consulta a BD

    producto = PRODUCTOS_DB.get(producto_id)
    if not producto:
        print(f" [ERROR] Producto {producto_id} no existe en la BD")
        return None

    # Almacenar en caché con TTL
    r.setex(clave_cache, CACHE_TTL, json.dumps(producto))
    print(f" [CACHE SET] Producto {producto_id} almacenado en Redis (TTL: {CACHE_TTL}s)")

    return producto
```

```
def invalidar_cache_producto(producto_id: int) -> None:
    """
    Invalida la caché de un producto específico.
    Se llama cuando el producto se modifica en la base de datos.
    """
    clave_cache = f"cache:producto:{producto_id}"
    r.delete(clave_cache)

if __name__ == "__main__":
    print("=== Cache-Aside Pattern Demo (Python) ===\n")

    # Ejemplo 1: Cache MISS
    print("--- Ejemplo 1: Primera consulta (Cache MISS) ---")
    prod = obtener_producto_con_cache(2)
    print(f"Producto obtenido: {prod['nombre']} – {prod['precio']}€\n")

    # Ejemplo 2: Cache HIT
    print("--- Ejemplo 2: Segunda consulta (Cache HIT) ---")
    prod = obtener_producto_con_cache(2)
    print(f"Producto obtenido: {prod['nombre']} – {prod['precio']}€\n")

    # Ejemplo 3: Invalidación
    print("--- Ejemplo 3: Invalidación de caché ---")
    invalidar_cache_producto(2)
    print("Caché del producto 2 eliminado.\n")

    # Ejemplo 4: Tras invalidar
    print("--- Ejemplo 4: Tras invalidar (Cache MISS) ---")
    prod = obtener_producto_con_cache(2)
    print(f"Producto obtenido: {prod['nombre']} – {prod['precio']}€\n")

    print("=== Demo completada ===")
```

4.3 Ejecutar el ejemplo

Terminal – Ejecutar

```
# Ejecutar el script
python cache_aside.py
```

Salida esperada:

```
=== Cache-Aside Pattern Demo (Python) ===

--- Ejemplo 1: Primera consulta (Cache MISS) ---
[CACHE MISS] Producto 2 no está en caché. Consultando BD...
[CACHE SET] Producto 2 almacenado en Redis (TTL: 300s)
Producto obtenido: Monitor 4K 27" – 449.95€

--- Ejemplo 2: Segunda consulta (Cache HIT) ---
[CACHE HIT] Producto 2 encontrado en Redis
Producto obtenido: Monitor 4K 27" – 449.95€

--- Ejemplo 3: Invalidación de caché ---
Caché del producto 2 eliminado.

--- Ejemplo 4: Tras invalidar (Cache MISS) ---
[CACHE MISS] Producto 2 no está en caché. Consultando BD...
[CACHE SET] Producto 2 almacenado en Redis (TTL: 300s)
Producto obtenido: Monitor 4K 27" – 449.95€

=== Demo completada ===
```

Comparativa C# vs Python: Ambas implementaciones siguen la misma lógica. La diferencia principal es el cliente Redis: StackExchange.Redis en C# (asíncrono con `async/await`) y `redis-py` en Python (síncrono por defecto, con soporte `async` opcional). Elige según tu stack tecnológico.

Paso 5: Invalidación de caché

La invalidación de caché es fundamental para mantener la coherencia entre la base de datos y la caché. Cuando un dato se modifica en la BD, debemos eliminar su correspondiente entrada en Redis para que la próxima lectura obtenga datos frescos.

5.1 Invalidar con DEL

Redis CLI – Eliminar caché de un producto

```
# Verificar que el caché del producto 2 existe
EXISTS cache:producto:2
```

Salida esperada:

Resultado

```
(integer) 1
```

Redis CLI – Eliminar la clave de caché

```
# Eliminar la entrada de caché del producto 2
# Esto se haría tras un UPDATE en la base de datos
DEL cache:producto:2
```

Salida esperada:

Resultado

```
(integer) 1
```

5.2 Verificar que la invalidación funcionó

Redis CLI – Confirmar eliminación

```
# Verificar que ya no existe en caché
EXISTS cache:producto:2
```

Salida esperada:

Resultado

```
(integer) 0
```

5.3 Invalidación de múltiples productos

Redis CLI — Eliminar varias claves de caché

```
# Eliminar caché de varios productos a la vez  
# Útil cuando se actualiza una categoría de productos  
DEL cache:producto:1 cache:producto:2 cache:producto:3
```

Salida esperada:

Resultado

```
(integer) 3
```

Estrategias de invalidación:

- **Invalidación inmediata:** Eliminar la clave de caché justo después de cada escritura en la BD. Simple pero genera más cache misses.
- **Actualización write-through:** Actualizar tanto la BD como la caché en la misma operación. Más complejo pero garantiza coherencia.
- **TTL como respaldo:** Aunque se invalide manualmente, el TTL actúa como mecanismo de seguridad contra datos obsoletos.

¿Cuándo invalidar? Invalida la caché siempre que se modifiquen datos en la base de datos: UPDATE, DELETE o INSERT de datos dependientes. También es recomendable invalidar periódicamente mediante el TTL para casos donde la BD se modifica externamente.

Preguntas de reflexión

Antes de continuar con el siguiente laboratorio, reflexiona sobre las siguientes preguntas:

1. ¿Qué problema resuelve el patrón Cache-Aside?

Piensa en los tiempos de respuesta de una consulta a MariaDB frente a una lectura en Redis. ¿Cuántas órdenes de magnitud de diferencia hay? ¿En qué escenarios sería más beneficioso?

2. ¿Por qué se usa TTL en la caché aunque ya invalidamos manualmente?

Considera el caso de que la BD se modifique por otro sistema o por un administrador. ¿Qué pasaría con los datos en caché si no tuviésemos TTL? ¿Qué valor de TTL seleccionarías para datos de catálogo de productos?

3. ¿Qué es el "cache stampede" y cómo lo evitarías?

Imagina que miles de usuarios solicitan simultáneamente un producto cuyo caché acaba de expirar. ¿Qué ocurriría en la base de datos? ¿Cómo podrías mitigar este problema en la implementación de C# o Python?

4. ¿Cuándo NO deberías usar Cache-Aside?

Considera datos que cambian con mucha frecuencia (por ejemplo, precio de una acción en bolsa) o datos con requisitos de consistencia fuerte (transferencias bancarias). ¿Sería apropiado usar caché en estos casos? ¿Qué alternativas existen?

5. ¿Cómo implementarías la invalidación de caché en un microservicio distribuido?

Si tienes múltiples instancias del servicio corriendo en diferentes servidores, ¿cómo garantizas que todas invaliden la misma clave de caché? ¿Qué patrones de mensajería (pub/sub, event sourcing) podrías utilizar?

Parte 2 Rate Limiting



Objetivo

Implementar **rate limiting** con Redis utilizando los comandos INCR y Sorted Sets. Al finalizar, serás capaz de controlar el número de peticiones que un usuario puede realizar en un período de tiempo dado, tanto con ventanas fijas como con ventanas deslizantes.

Al finalizar este laboratorio serás capaz de:

- Implementar rate limiting con ventana fija usando INCR y EXPIRE.
- Simular peticiones de múltiples usuarios con DataGrip.
- Implementar rate limiting con ventana deslizante usando Sorted Sets.
- Crear un ejemplo completo de rate limiting en C# con StackExchange.Redis.
- Crear un ejemplo completo de rate limiting en Python con redis-py.

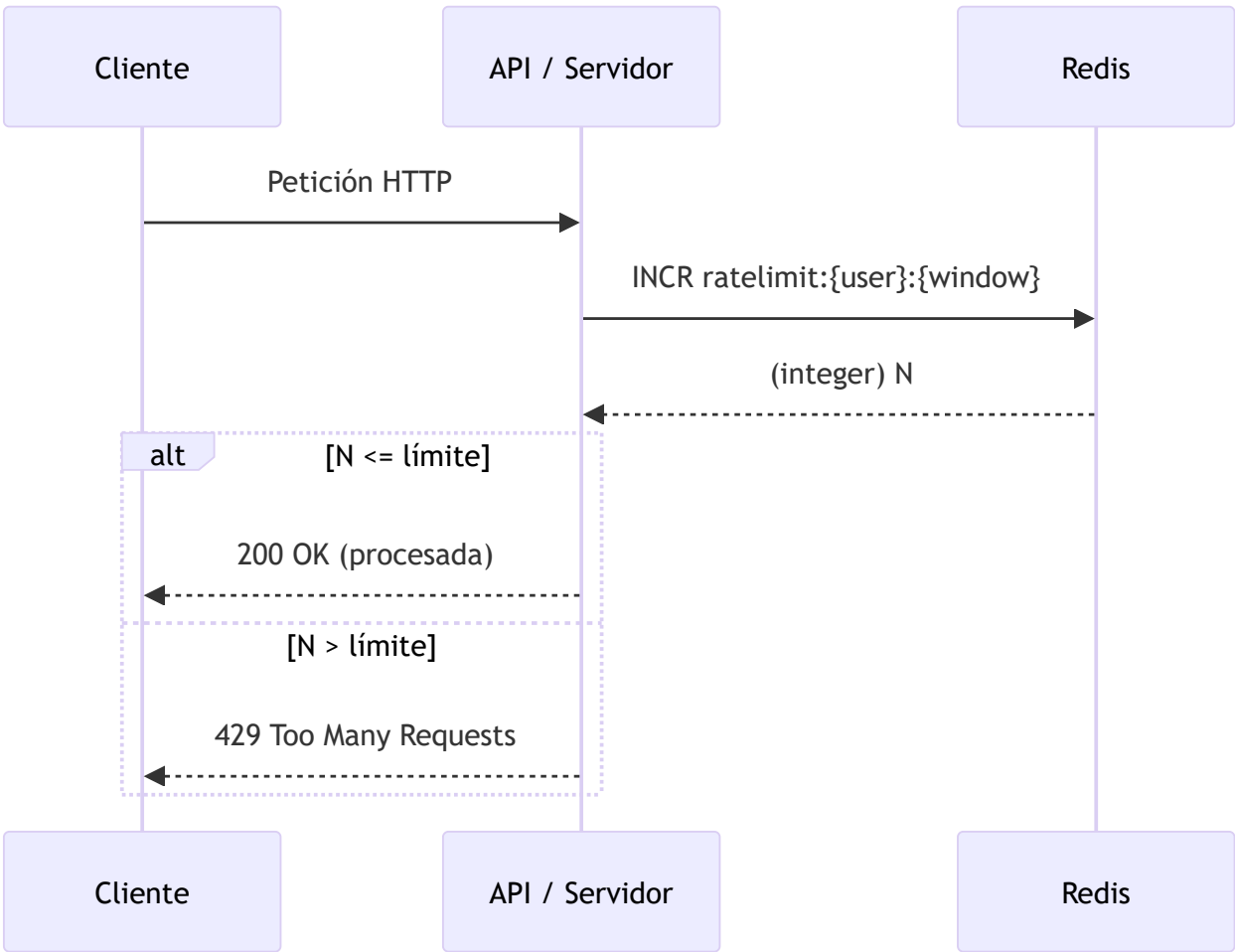
Requisitos previos

Componente	Versión mínima	Comprobación
Docker Desktop	4.x	<code>docker --version</code>
JetBrains DataGrip	2024.x	Abrir la aplicación
.NET SDK	10.x	<code>dotnet --version</code>
Python	3.12+	<code>python --version</code>
Práctica PR1 completada	—	Redis ejecutándose en localhost:6379
Práctica PR2 completada	—	Estructuras de datos comprendidas

Nota: Asegúrate de que Redis está en ejecución antes de comenzar. Puedes verificarlo con `docker ps` o ejecutando `redis-cli PING`.

Arquitectura del laboratorio

El siguiente diagrama muestra cómo funciona el rate limiting con Redis. Cada petición del cliente se intercepta, se incrementa un contador atómico en Redis y se decide si se permite o se bloquea:



Concepto clave: INCR es una operación atómica que incrementa un contador de forma segura en concurrencia. Si la clave no existe, la crea con valor 1. Esto lo hace ideal para rate limiting en entornos donde múltiples peticiones concurrentes acceden al mismo contador.

Paso 1: Rate limiting simple con INCR

El enfoque más básico utiliza una **ventana fija de tiempo**: se cuenta el número de peticiones en un intervalo determinado (por ejemplo, un minuto) y se bloquea al superar el límite. Se combina INCR con EXPIRE para que el contador se reinicie automáticamente.

redis-cli: Rate limiting con INCR + EXPIRE

```
# Rate limiting simple: máximo 100 peticiones por minuto
# Formato de clave: ratelimit:{user_id}:{timestamp_minuto}

# Simular petición del usuario 1000
# Creamos la clave con valor 0 antes de incrementar
SET ratelimit:1000:1726500000 0
# Output: OK

# Incrementar el contador (primera petición)
INCR ratelimit:1000:1726500000
# Output: (integer) 1

# Simular 99 peticiones más (en producción sería un bucle en la aplicación)
# Por simplicidad, usamos INCRBY para saltar al límite
INCRBY ratelimit:1000:1726500000 99
# Output: (integer) 100

# Establecer TTL de 60 segundos
# La clave expira automáticamente tras 1 minuto
EXPIRE ratelimit:1000:1726500000 60
# Output: (integer) 1

# Verificar TTL restante
TTL ratelimit:1000:1726500000
# Output: (integer) 58

# Petición 101 → bloqueada (supera el límite de 100)
INCR ratelimit:1000:1726500000
# Output: (integer) 101
# La aplicación debería devolver: 429 Too Many Requests
```

Salida esperada completa:

```
redis> SET ratelimit:1000:1726500000 0
OK

redis> INCR ratelimit:1000:1726500000
(integer) 1

redis> INCRBY ratelimit:1000:1726500000 99
(integer) 100

redis> EXPIRE ratelimit:1000:1726500000 60
(integer) 1

redis> TTL ratelimit:1000:1726500000
(integer) 58

redis> INCR ratelimit:1000:1726500000
(integer) 101
```

¿Por qué INCR en lugar de GET + SET? Porque INCR es una operación atómica en Redis. Si dos peticiones concurrentes ejecutan GET + SET al mismo tiempo, ambas podrían leer el mismo valor y escribir el mismo incremento, perdiendo una actualización. Con INCR, Redis garantiza que el incremento es secuencial, evitando condiciones de carrera.

Paso 2: Rate limiting con DataGrip

En este paso simularemos el flujo completo de rate limiting desde DataGrip, realizando 10 peticiones de 2 usuarios distintos para comprobar que cada usuario tiene su propio contador independiente.

2.1 — Peticiones del usuario 1000

DataGrip: Simular 10 peticiones del usuario 1000

```
# Resetear el contador del usuario 1000
DEL ratelimit:1000:1726500000
# Output: (integer) 1

# Simular 10 peticiones del usuario 1000
INCR ratelimit:1000:1726500000
# Output: (integer) 1

INCR ratelimit:1000:1726500000
# Output: (integer) 2

INCR ratelimit:1000:1726500000
# Output: (integer) 3

INCR ratelimit:1000:1726500000
# Output: (integer) 4

INCR ratelimit:1000:1726500000
# Output: (integer) 5

INCR ratelimit:1000:1726500000
# Output: (integer) 6

INCR ratelimit:1000:1726500000
# Output: (integer) 7

INCR ratelimit:1000:1726500000
# Output: (integer) 8

INCR ratelimit:1000:1726500000
# Output: (integer) 9

INCR ratelimit:1000:1726500000
# Output: (integer) 10

# Establecer TTL de 60 segundos
EXPIRE ratelimit:1000:1726500000 60
# Output: (integer) 1
```

2.2 — Peticiones del usuario 2000

DataGrip: Simular 10 peticiones del usuario 2000

```
# Simular 10 peticiones del usuario 2000
# Cada usuario tiene su propio contador, identificado por su user_id
INCR ratelimit:2000:1726500000
# Output: (integer) 1

INCR ratelimit:2000:1726500000
# Output: (integer) 2

INCR ratelimit:2000:1726500000
# Output: (integer) 3

INCR ratelimit:2000:1726500000
# Output: (integer) 4

INCR ratelimit:2000:1726500000
# Output: (integer) 5

INCR ratelimit:2000:1726500000
# Output: (integer) 6

INCR ratelimit:2000:1726500000
# Output: (integer) 7

INCR ratelimit:2000:1726500000
# Output: (integer) 8

INCR ratelimit:2000:1726500000
# Output: (integer) 9

INCR ratelimit:2000:1726500000
# Output: (integer) 10

# Establecer TTL de 60 segundos
EXPIRE ratelimit:2000:1726500000 60
# Output: (integer) 1
```

2.3 — Verificar contadores

DataGrip: Consultar estado de ambos usuarios

```
# Verificar contador del usuario 1000
GET ratelimit:1000:1726500000
# Output: "10"

# Verificar contador del usuario 2000
GET ratelimit:2000:1726500000
# Output: "10"

# Verificar TTL de ambos contadores
TTL ratelimit:1000:1726500000
# Output: (integer) 55

TTL ratelimit:2000:1726500000
# Output: (integer) 55

# Simular una petición más del usuario 1000
INCR ratelimit:1000:1726500000
# Output: (integer) 11
# → Permitido (dentro del límite de 100)

# Verificar que ambos contadores son independientes
MGET ratelimit:1000:1726500000 ratelimit:2000:1726500000
# Output: "11" → "10"
```

Independencia de contadores: Cada usuario tiene su propio contador, identificado por su `user_id` en la clave. Esto permite gestionar límites independientes por usuario sin interferencias entre ellos. El formato de clave `ratelimit:{user_id}:{ventana}` es fundamental para esta separación.

Paso 3: Rate limiting avanzado con Sorted Sets

El enfoque de ventana fija tiene un problema: si un usuario realiza 100 peticiones en el último segundo del minuto y 100 en el primero del siguiente, en la práctica ha realizado 200 peticiones en 2 segundos, pero cada ventana las cuenta por separado. La **ventana deslizante** resuelve esto utilizando Sorted Sets, donde cada petición se almacena como un miembro con el timestamp como score.

redis-cli: Rate limiting con ventana deslizante (Sorted Set)

```
# Rate limiting con ventana deslizante (Sorted Set)
# Cada petición se almacena como miembro del Sorted Set
# Score = timestamp en milisegundos

# Limpiar datos anteriores
DEL ratelimit:1000:window
# Output: (integer) 1

# Simular 5 peticiones con timestamps incrementales
# Cada petición tiene un timestamp distinto (score)
ZADD ratelimit:1000:window 1726500000000 "req:1"
# Output: (integer) 1

ZADD ratelimit:1000:window 1726500001000 "req:2"
# Output: (integer) 1

ZADD ratelimit:1000:window 1726500002000 "req:3"
# Output: (integer) 1

ZADD ratelimit:1000:window 1726500003000 "req:4"
# Output: (integer) 1

ZADD ratelimit:1000:window 1726500004000 "req:5"
# Output: (integer) 1

# Contar todas las peticiones en la ventana
ZCARD ratelimit:1000:window
# Output: (integer) 5

# Obtener las peticiones de los últimos 60 segundos
ZRANGEBYSCORE ratelimit:1000:window 1726500000000 1726500060000
# Output: "req:1" → "req:2" → "req:3" → "req:4" → "req:5"

# Eliminar peticiones fuera de la ventana (antiguas)
ZREMRANGEBYSCORE ratelimit:1000:window 0 1726499940000
# Output: (integer) 0 # No hay peticiones antiguas
```

Salida esperada completa:

```
redis> DEL ratelimit:1000:window
(integer) 1

redis> ZADD ratelimit:1000:window 1726500000000 "req:1"
(integer) 1

redis> ZADD ratelimit:1000:window 1726500001000 "req:2"
(integer) 1

redis> ZADD ratelimit:1000:window 1726500002000 "req:3"
(integer) 1

redis> ZADD ratelimit:1000:window 1726500003000 "req:4"
(integer) 1

redis> ZADD ratelimit:1000:window 1726500004000 "req:5"
(integer) 1

redis> ZCARD ratelimit:1000:window
(integer) 5

redis> ZRANGEBYSCORE ratelimit:1000:window 1726500000000 1726500006000
"req:1" → "req:2" → "req:3" → "req:4" → "req:5"

redis> ZREMRANGEBYSCORE ratelimit:1000:window 0 1726499940000
(integer) 0
```

Ventana fija vs. deslizante: Con ventana fija, el contador se resetea al inicio de cada minuto. Con ventana deslizante, se cuentan las peticiones de los últimos 60 segundos en cualquier momento, lo que proporciona un control más preciso y evita picos de tráfico entre ventanas consecutivas.

Paso 4: C# con StackExchange.Redis

Implementación completa de rate limiting en C# utilizando StackExchange.Redis. Se muestra tanto el enfoque de ventana fija con INCR como el de ventana deslizante con Sorted Sets.

4.1 — Instalación de paquetes NuGet

Terminal: Instalar dependencia

```
# Instalar el cliente oficial de Redis para .NET  
dotnet add package StackExchange.Redis
```

4.2 — Código completo

```
Program.cs - Rate Limiting en C#
```

```
// .NET 10 + C# 14: Rate Limiting con Redis
using StackExchange.Redis;

// Conexión a Redis (pool de conexiones gestionado por StackExchange.Redis)
using var redis = await ConnectionMultiplexer.ConnectAsync("localhost:6379");
var db = redis.GetDatabase();

// -----
// ENFOQUE 1: Ventana fija con INCR + EXPIRE
// -----

// Rate limiting por usuario: máximo 100 peticiones por minuto
async Task<bool> IsRateLimitedFixedWindowAsync(int userId, int maxRequests = 100)
{
    // Generar clave basada en el minuto actual
    // Dividimos el timestamp entre 60 para obtener la "ventana" de minuto
    var minute = DateTimeOffset.UtcNow.ToUnixTimeSeconds() / 60;
    var key = $"ratelimit:{userId}:{minute}";

    // INCR: incrementa atómicamente el contador
    // Si la clave no existe, la crea con valor 1
    var count = await db.StringIncrementAsync(key);

    // Si es la primera petición, establecer TTL de 60 segundos
    // Esto garantiza que la clave se elimine automáticamente
    if (count == 1)
    {
        await db.KeyExpireAsync(key, TimeSpan.FromMinutes(1));
    }

    // Verificar límite
    return count > maxRequests;
}

// -----
// ENFOQUE 2: Ventana deslizante con Sorted Sets
// -----

// Rate limiting con ventana deslizante: cuenta peticiones de los últimos 60 segundos
async Task<bool> IsRateLimitedSlidingWindowAsync(int userId, int maxRequests = 100)
{
    var key = $"ratelimit:{userId}:window";
    var now = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    var windowStart = now - 60_000; // 60 segundos atrás en milisegundos

    // Eliminar peticiones fuera de la ventana (antiguas)
    await db.SortedSetRemoveRangeByScoreAsync(key, 0, windowStart);

    // Contar peticiones actuales en la ventana
    var count = await db.SortedSetLengthAsync(key);
}
```

```

    if (count >= maxRequests)
    {
        return true; // Bloqueado
    }

    // Registrar la petición actual con timestamp como score
    var memberId = $"req:{Guid.NewGuid():N}";
    await db.SortedSetAddAsync(key, memberId, now);

    // Establecer TTL de 60 segundos para limpieza automática
    await db.KeyExpireAsync(key, TimeSpan.FromSeconds(60));

    return false; // Permitido
}

// -----
// EJEMPLO DE USO
// -----

var userId = 1000;

// Probar ventana fija
if (await IsRateLimitedFixedWindowAsync(userId))
{
    Console.WriteLine("429 Too Many Requests (ventana fija)");
}
else
{
    Console.WriteLine("200 OK (ventana fija)");
}

// Probar ventana deslizante
if (await IsRateLimitedSlidingWindowAsync(userId))
{
    Console.WriteLine("429 Too Many Requests (ventana deslizante)");
}
else
{
    Console.WriteLine("200 OK (ventana deslizante)");
}

```

Salida esperada:

Salida de la aplicación

```

200 OK (ventana fija)
200 OK (ventana deslizante)

```

¿Por qué `StringIncrementAsync`? Es la versión asíncrona de INCR. Realiza una sola operación de red atómica, lo que lo hace ideal para rate limiting en entornos de alta concurrencia. No hay riesgo de condiciones de carrera entre peticiones concurrentes.

Paso 5: Alternativa en Python con redis-py

Este paso es opcional. Se muestra la misma implementación en Python para quienes prefieran este lenguaje o quieran comparar ambas aproximaciones.

5.1 — Instalación de dependencias

Terminal: Instalar paquete

```
# Instalar el cliente oficial de Redis para Python  
pip install redis
```

5.2 — Código completo

```
rate_limiting.py - Rate Limiting en Python
```

```

# redis-py: Rate Limiting con Redis
import redis
import time

# Conexión a Redis
# decode_responses=True: convierte bytes a strings automáticamente
r = redis.Redis(host='localhost', port=6379, decode_responses=True)

# -----
# ENFOQUE 1: Ventana fija con INCR + EXPIRE
# -----

def is_rate_limited_fixed_window(user_id: int, max_requests: int = 100) -> bool:
    """Verificar si el usuario ha excedido el límite (ventana fija)."""
    # Generar clave basada en el minuto actual
    # Dividimos el timestamp entre 60 para obtener la "ventana" de minuto
    minute = int(time.time() / 60)
    key = f"ratelimit:{user_id}:{minute}"

    # INCR: incrementa atómicamente el contador
    # Si la clave no existe, la crea con valor 1
    count = r.incr(key)

    # Si es la primera petición, establecer TTL de 60 segundos
    # Esto garantiza que la clave se elimine automáticamente
    if count == 1:
        r.expire(key, 60)

    # Verificar límite
    return count > max_requests

# -----
# ENFOQUE 2: Ventana deslizante con Sorted Sets
# -----

def is_rate_limited_sliding_window(user_id: int, max_requests: int = 100) -> bool:
    """Verificar si el usuario ha excedido el límite (ventana deslizante)."""
    key = f"ratelimit:{user_id}:window"
    now = int(time.time() * 1000) # Timestamp en milisegundos
    window_start = now - 60_000 # 60 segundos atrás

    # Eliminar peticiones fuera de la ventana (antiguas)
    r.zremrangebyscore(key, 0, window_start)

    # Contar peticiones actuales en la ventana
    count = r.zcard(key)

    if count >= max_requests:
        return True # Bloqueado

```

```

# Registrar la petición actual con timestamp como score
member_id = f"req:{int(time.time()) * 1000}"
r.zadd(key, {member_id: now})

# Establecer TTL de 60 segundos para limpieza automática
r.expire(key, 60)

return False # Permitido

# -----
# EJEMPLO DE USO
# -----

user_id = 1000

# Probar ventana fija
if is_rate_limited_fixed_window(user_id):
    print("429 Too Many Requests (ventana fija)")
else:
    print("200 OK (ventana fija)")

# Probar ventana deslizante
if is_rate_limited_sliding_window(user_id):
    print("429 Too Many Requests (ventana deslizante)")
else:
    print("200 OK (ventana deslizante)")

```

Salida esperada:

Salida de la aplicación

```

200 OK (ventana fija)
200 OK (ventana deslizante)

```

decode_responses=True: Sin esta opción, Redis devuelve datos en formato bytes. Con ella, redis-py los convierte automáticamente a strings de Python, simplificando el manejo de la respuesta.

Preguntas de reflexión

Antes de continuar con el siguiente laboratorio, reflexiona sobre las siguientes preguntas:

1. ¿Por qué es mejor usar INCR que GET + SET para rate limiting?

Piensa en la atomicidad de las operaciones. ¿Qué problema surgiría si dos peticiones concurrentes ejecutaran GET + SET al mismo tiempo?

2. ¿Qué problema resuelve la ventana deslizante frente a la ventana fija?

Considera un usuario que realiza 100 peticiones en el último segundo de un minuto y 100 en el primero del siguiente. ¿Cuántas peticiones reales ha realizado en 2 segundos? ¿Cómo lo maneja cada enfoque?

3. ¿Cómo manejarías rate limiting en una arquitectura de microservicios?

Si tienes múltiples instancias del servidor, ¿cómo garantizas que el contador sea global? ¿Qué pasaría si cada instancia tuviera su propio contador local?

4. ¿Qué estrategia de *fallback* aplicarías si Redis cae?

¿Permitirías las peticiones sin rate limiting (*fail-open*) o las bloquearías (*fail-closed*)? ¿Qué factores determinan esta decisión?

5. ¿Cómo configurarías rate limiting para diferentes endpoints?

Piensa en endpoints de lectura vs. escritura. ¿Usarías el mismo límite para ambos? ¿Cómo definirías las claves de Redis para distinguirlos?



Arquitectura de Datos

4.º Curso, Grado en Ingeniería Informática

Curso académico 2026/2027

Human Language and Accessibility Technologies Group (HULAT).

Departamento de Informática

Universidad Carlos III de Madrid